SPECS · ENTERPRISE ENTRY

Server-Side Function Tools with AI Runners

Every proposal that begins **“first we replace”** dies in the same meeting. Bounded server-side functions that **resolve authority per call**, **record themselves**, and give an AI runner capability — with **no migration to own**.

For: **teams that need to add agent capability to an enterprise stack they are not allowed to replace.**

Premise: **anything requiring a migration will not survive review. This does not require one.**

The entry problem

Every proposal that begins "first we replace" dies in the same meeting. Not because the proposal is wrong — often it is correct — but because replacement carries migration risk, and migration risk needs an owner, and nobody volunteers to own it against a system that currently works.

So the practical question is not *what should this stack be.* It is: **what can be added to it, this quarter, without asking anyone to move anything.**

The answer that clears review is a **server-side function tool**: a small, bounded operation that runs beside the existing system rather than inside it, resolves authority on every call, writes its own record, and returns a typed result an agent can act on.

What a function tool actually is

Four properties. Miss any one and it stops being safe to expose.

1. Bounded. It does one thing with a stated scope. Not "update the customer" but "record a consent decision for this subject, for this purpose, in this market." A tool whose blast radius cannot be described in a sentence cannot be governed.

2. Authority resolved per call. The caller presents a scoped token; the server resolves what that subject may do *at that moment* — scope, cap, expiry, consent state — before the effect happens. Not at authentication. At the effect.

3. Self-recording. Every invocation writes an append-only row: actor, action, subject, scope, jurisdiction, decision, timestamp, previous-row hash. **Including refusals.** A denial is evidence the control was live.

4. Typed and idempotent. A declared input and output shape, plus an idempotency key so a retry produces one effect rather than two. Every real delivery channel is at-least-once; agents retry more than people do.

Why this clears an enterprise review

The objections a security or architecture board actually raises, and what this answers:

OBJECTION	ANSWER
"It touches our system of record."	It does not. It runs beside it and writes its own ledger.
"What can it do if it misbehaves?"	Exactly what its scope permits, capped and expiring. The boundary is enforced where the effect lands.
"Can we turn it off?"	One revocation, affecting nothing else. Per-actor identity means no shared credential to break.
"Who approved this action?"	The mandate names an accountable human; the ledger row carries the mandate.
"What happens when it fails?"	It refuses, and the refusal is recorded. Fail-closed is the default, not the fallback.
"How long is the migration?"	There is no migration.

That last row is the one that gets it funded. Everything else is a conversation; a migration is a programme.

Where AI runners fit

An **AI runner** is an agent that invokes these tools rather than driving a UI. The distinction matters more than it sounds.

Agents that operate through screens — clicking, scraping, filling forms — inherit whatever the screen permits, which is usually everything the logged-in human can do. There is no way to bound them, no way to attribute them separately, and no record beyond a session log. That is automation without authority.

An agent invoking function tools is the opposite shape:

- It holds a **scoped token**, not a credential.
- Each call resolves the caller's entitlement **server-side** before acting.

- A call outside scope, over cap, or past expiry is **refused**, not logged-and-permitted.
- The refusal is recorded, as is the grant.

Practically, this means the agent needs no understanding of your permission model. It needs a token that is already bounded, and a server that resolves the claim on every call. A misbehaving agent becomes a non-event rather than an incident — because the boundary lives where the effect happens, not where the agent was configured.

Exposure is conventional: an MCP server over Streamable HTTP, tools declared with their input schemas, the caller's claim resolved per tool call.

The sequence, which is the part most teams invert

Instrument first. Automate second.

A self-correcting loop cannot correct what it cannot observe. An agent that writes, tests and revises is only as good as the signal it acts on — put it in an environment where nobody can say which change reached which system, and it will iterate confidently on the wrong thing, faster than anyone can catch it.

So the order is:

1. **Identify the gaps.** Which decisions currently happen with no record? Which credentials are shared? Which values are overwritten that someone may ask about later?
2. **Wrap the first one in a function tool.** One operation, bounded, recording, refusing when it should.
3. **Point a runner at it.** Now the loop has both a capability and a record of using it.

4. **Repeat.** Each tool is independently useful and independently revocable, so the work compounds without a big-bang date.

This is why the two agendas turn out to be one. The instrumentation that satisfies an evidentiary obligation is the same instrumentation an AI partnership needs to be trustworthy. One piece of work, two payoffs.

What it costs to start

Less than the meeting about it. The substrate is already present in most modern stacks or free on signup: a managed backend for the ledger and entitlements, edge compute for the function surface and signing, source control for the change record.

One engineer can stand up an entitlement model, a mandate, a hash-chained ledger and a first tool inside their own scope, prove it against real calls, and present it as a working specification rather than an infrastructure request. That converts the procurement conversation from *whether this works* to *scale and support* — a different meeting with a different outcome.

The short version

- **Do not propose replacement.** Propose addition.
- **A function tool is bounded, authority-resolved-per-call, self-recording, and idempotent.** All four, or it is not safe to expose.
- **Agents should invoke tools, not drive screens.** Screens carry the human's full authority; tools carry a bounded token.
- **Record refusals.** They are the evidence your controls were operating.

- **Instrument before you automate.** A loop without provenance is not resilience — it is unattributable drift, accelerating.

Companion documents: AI Trust Framework Requirements for the ten requirements and the additive REST changes; Agent Authority — Technical Brief for the mandate model and endpoints; The Trust Framework for the non-technical case.
